

Документация для практических заданий

Документация для решения практических заданий

Этот файл — краткая карта документации для 16 практических задач из тренажёра. Сначала смотри общий блок, затем раздел по конкретной задаче.

Базовый набор

- Python: структуры данных
<https://docs.python.org/3/tutorial/datastructures.html>
Нужны списки, словари, множества, list comprehension, использование списка как стека.
- Python: встроенные функции
<https://docs.python.org/3/library/functions.html>
Нужны sum, len, min, max, range, isinstance, zip, enumerate, sorted.
- Python: классы
<https://docs.python.org/3/tutorial/classes.html>
Нужны классы, методы экземпляра, наследование, итераторы, область видимости.
- Python: модель данных
<https://docs.python.org/3/reference/datamodel.html>
Нужны магические методы __len__, __iter__, __next__, __enter__, __exit__.
- Python: регулярные выражения
<https://docs.python.org/3/library/re.html>
Нужны для задачи с HTML-подобными тегами.
- Python: functools
<https://docs.python.org/3/library/functools.html>
Нужны lru_cache и wraps.
- Python: время
<https://docs.python.org/3/library/time.html>
Нужен perf_counter для измерения производительности.

- Python: даты и время
<https://docs.python.org/3/library/datetime.html>
Нужен `datetime.strptime` и работа с датами.
- Python: контейнеры
<https://docs.python.org/3/library/collections.html>
Нужен `defaultdict`; также полезно знать `deque`.
- Python: `with` и контекстные менеджеры
https://docs.python.org/3/reference/compound_stmts.html#with
Нужно для `FileWriter`.
- Python: JSON
<https://docs.python.org/3/library/json.html>
Нужно для задач с JSON-объектами и словарями.
- Python: бинарный поиск в стандартной библиотеке
<https://docs.python.org/3/library/bisect.html>
Полезно для понимания границ и поиска позиции.
- Python: сортировка
<https://docs.python.org/3/howto/sorting.html>
Нужны `sorted`, `list.sort`, ключи сортировки и стабильность сортировки.
- Сложность операций Python-контейнеров
<https://wiki.python.org/moin/TimeComplexity>
Нужна для оценки `list`, `dict`, `set`, стека, поиска, вставок.

Алгоритмы и структуры данных

1. Ближайшая большая высота

Что читать: - Python data structures: раздел про списки как стек. - Сложность операций Python-контейнеров.

Что нужно понять: - монотонный стек индексов; - почему каждый элемент кладётся и снимается со стека не более одного раза; - почему решение работает за $O(n)$.

Минимальный план решения: 1. Идти по массиву справа налево. 2. Удалять из стека все здания не выше текущего. 3. Если стек не пуст, верхушка — ближайшее большее здание справа. 4. Записать расстояние между индексами.

2. Самый частотный символ

Что читать: - Python data structures: словари и множества. - Built-in functions: `max`, `sum`.

Что нужно понять: - нормализация регистра через `lower` или `casefold`; - фильтрация только букв; - подсчёт частот через `dict`; - обработка пустой строки.

Минимальный план решения: 1. Пройти по строке. 2. Учитывать только латинские буквы. 3. Считать частоты в словаре. 4. Найти максимум. 5. Сложить частоты всех символов, у которых частота равна максимуму.

3. k-й элемент двух отсортированных массивов

Что читать: - `bisect` как справку по идее бинарного поиска. - Python data structures: индексация списков. - TimeComplexity.

Что нужно понять: - почему нельзя сливать массивы; - как отбрасывать часть одного массива; - почему шаг уменьшает k ; - граничные случаи: один массив пустой, $k == 1$.

Минимальный план решения: 1. Хранить текущие индексы в обоих массивах. 2. Сравнивать элементы около $k // 2$. 3. Отбрасывать часть массива, которая точно меньше k -го элемента. 4. Повторять до базового случая.

4. Слияние и сортировка слиянием

Что читать: - Python sorting HOWTO. - Python data structures: списки и срезы.

Что нужно понять: - слияние двумя указателями; - рекурсивное деление массива; - разница между собственным `merge sort` и встроенной сортировкой; - память $O(n)$ и время $O(n \log n)$.

Минимальный план решения: 1. Реализовать `merge_sorted_times`. 2. В `merge_sort_times` делить список пополам. 3. Рекурсивно сортировать половины. 4. Объединять через `merge_sorted_times`.

5. Бинарный поиск и метод бисекции

Что читать: - `bisect`. - Built-in functions.

Что нужно понять: - поиск первого вхождения через смещение правой границы; - инварианты `left`, `right`, `mid`; - метод деления отрезка пополам для функции с разными знаками на концах; - точность `eps`.

Минимальный план решения: 1. Для массива хранить `answer` и искать левую границу. 2. Для корня проверять разные знаки на концах. 3. Пока длина отрезка больше `eps`, делить его пополам. 4. Оставлять половину, где знак меняется.

6. Проверка вложенности тегов

Что читать: - Python data structures: список как стек. - `re`: регулярные выражения.

Что нужно понять: - стек открывающих тегов; - различие `<tag>` и `{tag}`; - закрывающий тег должен совпадать с верхушкой стека; - пустой стек в конце означает корректную вложенность.

Минимальный план решения: 1. Регулярным выражением найти только теги. 2. Открывающий тег положить в стек. 3. Закрывающий сравнить с верхушкой. 4. При несовпадении вернуть `False`. 5. В конце проверить, что стек пуст.

Python

7. Класс DataKeeper

Что читать: - Python classes. - Data model: `__len__`. - Built-in functions: `sum`, `len`.

Что нужно понять: - отличие метода экземпляра, `@staticmethod`, `@classmethod`; - `@property`; - сохранение порядка уникальных элементов; - создание объекта из строки.

Минимальный план решения: 1. В `__init__` сохранить `source`. 2. Построить `distinct` через `set` для проверки повторов. 3. Реализовать `sum_distinct`. 4. Реализовать `calc_average`. 5. Реализовать `from_comma_string`. 6. Реализовать `__len__`.

8. Генератор, рекурсия, производительность

Что читать: - Python classes/tutorial: итераторы и генераторы. - `functools`: `lru_cache`. - `time`: `perf_counter`.

Что нужно понять: - `yield`; - экспоненциальная сложность простой рекурсии; - мемоизация; - честное измерение времени.

Минимальный план решения: 1. Написать генератор последовательности Люка. 2. Написать простую рекурсию. 3. Написать рекурсию с `lru_cache`. 4. Замерить время каждого способа через `perf_counter`.

9. Класс-итератор TableMapper

Что читать: - Python classes. - Data model: `__iter__`, `__next__`. - Data structures: словари и `zip`.

Что нужно понять: - объект-итератор возвращает сам себя из `__iter__`; - `__next__` должен бросать `StopIteration`; - некорректные строки нужно пропускать.

Минимальный план решения: 1. Сохранить заголовки из первой строки. 2. Хранить текущий индекс. 3. В `__next__` идти до корректной строки. 4. Возвращать `dict(zip(fields, row))`.

10. Анализ временных рядов

Что читать: - `datetime`. - `collections.defaultdict`. - Sorting HOWTO.

Что нужно понять: - парсинг даты через `datetime.strptime`; - группировка по полю `group`; - сортировка внутри группы по дате; - скользящее среднее по окну 3.

Минимальный план решения: 1. Преобразовать `dt` в дату и поля `yr`, `mo`, `dy`. 2. Сгруппировать записи. 3. Отсортировать каждую группу по дате. 4. Для индексов от 1 до `len-2` считать среднее соседей.

11. Декоратор `trace_calls`

Что читать: - `functools.wraps`. - `time.perf_counter`. - Python classes/functions: функции как объекты.

Что нужно понять: - декоратор возвращает `wrapper`; - `wraps` сохраняет имя и метаданные функции; - частоты элементов можно считать словарём; - порядок сохраняется проходом по исходному списку.

Минимальный план решения: 1. Написать `trace_calls(func)`. 2. В `wrapper` замерить время до и после вызова. 3. Вывести имя, аргументы, результат и время. 4. Для `extract_unique` посчитать частоты и вернуть элементы с частотой 1.

12. Наследование и полиморфизм

Что читать: - Python classes: наследование. - Built-in functions: `sum`. - Модуль `math` в официальной документации при необходимости для `pi`.

Что нужно понять: - базовый класс задаёт интерфейс; - дочерние классы переопределяют метод; - полиморфизм: `sum_areas` не должен знать конкретный тип фигуры.

Минимальный план решения: 1. Сделать `Figure.get_area`. 2. Реализовать `Square`, `Disk`, `RightTriangle`. 3. В каждом классе переопределить `get_area`. 4. В `sum_areas` вызвать `get_area` у каждого объекта.

13. Рекурсивное уплощение словаря

Что читать: - Python data structures: словари и списки. - Built-in functions: `isinstance`, `enumerate`.

Что нужно понять: - рекурсия по вложенным структурам; - путь ключа строится постепенно; - для списка в ключ добавляется индекс.

Минимальный план решения: 1. Создать результирующий словарь. 2. Написать внутреннюю функцию `visit(value, path)`. 3. Если значение словарь — пройти по ключам. 4. Если список — пройти по индексам. 5. Иначе записать значение по текущему пути.

14. Контекстный менеджер `FileWriter`

Что читать: - `with` statement. - Data model: `__enter__`, `__exit__`. - Built-in `open`.

Что нужно понять: - `__enter__` возвращает объект, доступный в `with`; - `__exit__` вызывается всегда; - чтобы не подавлять исключение, нужно вернуть `False`; - количество байт лучше считать через `len(text.encode('utf-8'))`.

Минимальный план решения: 1. Открыть файл в `__enter__`. 2. Реализовать метод `write`. 3. В `__exit__` записать ошибку, если она была. 4. Закрыть файл и вывести количество байт.

15. Валидация и обработка данных

Что читать: - Python data structures. - Built-in functions. - JSON docs полезны для понимания формата словарей и структур данных.

Что нужно понять: - вход нужно валидировать до вычислений; - ошибки возвращаются в едином формате; - mode должен быть одним из допустимых значений.

Минимальный план решения: 1. Проверить, что values — непустой список. 2. Проверить, что все элементы int. 3. Проверить mode. 4. Выполнить операцию. 5. Вернуть {"status": "ok", "result": ...} или {"status": "fail", "message": ...}.

16. Класс GridAnalyzer

Что читать: - Python data structures: вложенные списки. - Built-in functions: min, max, range, len. - List comprehensions.

Что нужно понять: - двумерная матрица — список списков; - строка берётся по индексу; - столбец собирается проходом по строкам; - транспонирование меняет строки и столбцы местами.

Минимальный план решения: 1. Сохранить grid. 2. В fetch_row проверить индекс строки. 3. В fetch_col проверить индекс столбца для всех строк. 4. Для find_max и find_min пройти по всем элементам. 5. Для create_transpose собрать новую матрицу через двойной проход.